

# Final Project Report — Programming Fundamentals

University Name: National University of Computer and Emerging Sciences (FAST-NUCES)

Department: Department of Software Engineering (BSSE)

Course: Programming Fundamentals (Lab)

Project Title: Smart All In One Calculator

Submitted By: Muhammad Hasnain (Roll No. 25K-3029), Sohail Ahmed (Roll No. 25K-3015)

Submitted To: Ms. Ravia Ejaz

Semester: Fall 2025

Date: 23/11/2025

---

## Abstract

We built a Smart Console Calculator & AI Assistant in C that goes way beyond your typical calculator. Instead of just doing basic math, our program handles scientific calculations, converts units, plots graphs, and even includes an AI assistant—all from the console. We used the Shunting-yard algorithm to make sure operations follow proper order (BODMAS rules), and we worked with the Windows API to make the interface more interactive with colors and better visuals. One of the coolest parts is how we integrated Google's Gemini API through system calls, which lets users ask complex questions and even analyze images right from the console. This project really pushed us to work with data structures like stacks, handle strings properly, and connect to external APIs.

## 1. Introduction

When you're in your first semester of Computer Science, it's honestly hard to connect what you're learning—loops, structs, basic logic—to actual useful software. Most calculator projects students do are pretty

boring and don't do much more than add and subtract numbers. We wanted to build something that actually matters and shows how fundamental programming connects to real technology. Our calculator is designed for engineering students who need to solve complicated equations, see mathematical functions visualized (like parabolas drawn with ASCII characters), and get answers to conceptual questions through AI. We went from basic programming concepts to dealing with memory management, pointers, and the logic behind parsing math expressions.

## 2. Objectives

- Build solid mathematical logic: We used stacks to evaluate expressions with correct operator precedence (so  $2+3*4$  gives you 14, not 20).
- Make math visual: We created a coordinate system in the console that can plot functions like  $y = x^2$ .
- Bring in modern AI: We connected our C program to the internet so it can fetch answers from Google's Gemini AI.
- Make it look good: Using windows.h, we added console colors, controlled cursor positioning, and even added file selection windows.
- Really nail C fundamentals: This helped us get much better at structs, pointers, file I/O, and parsing strings.

---

## 3. System Design

### System Overview

The program runs in a continuous loop with a main menu that gives you four different modes to choose from. We used a "polled" input

method to avoid those annoying buffer errors and make sure switching between modes feels smooth.

How the program flows:

Start → Main Menu → Pick a Mode (Scientific / Converter / Graphing / AI) → Do Your Thing → See the Output → Go Back to Menu or Exit.

### **Algorithm (Scientific Mode)**

1. Input: You type in something like  $10 + 2 * 5$ .
2. Parsing: We clean up the string and remove spaces.
3. Conversion: The Shunting-yard algorithm converts from Infix notation (how humans write it) to Postfix notation (how computers like it) using a Character Stack.
4. Evaluation: The Postfix expression gets calculated using a Double Stack so we can handle decimal numbers.
5. Output: You see your answer.

### **Input & Output**

- Input: Math expressions, menu choices, questions for the AI, and image files.
- Output: Calculated answers, converted measurements, ASCII graphs, and AI responses.

---

## 4. Implementation

Language: C (C99 Standard)

Compiler/IDE: GCC / CLion / VS Code / Dev-C++

External Dependencies: windows.h (for UI), comdlg32 (for File Dialogs), libcurl / System Calls (for Network stuff).

## Key Features

- Parser-Based Calculator: Handles equations with brackets ( ) and decimal points.
- ASCII Graphing Engine: Plots x vs y coordinates dynamically right in the console.
- AI Vision & Chat: Sends images and text to Google's Gemini 2.5 Flash model and shows you what it says.
- Buffer-Safe Input: We made our own getLine() function because scanf() kept causing problems.

## Code Snippet (AI Request Logic)

```
// Building the command to send JSON data to Google Gemini API
sprintf(command,
    "curl -s -X POST
    \"https://generativelanguage.googleapis.com/v1beta/models/gemini-
    2.5-flash:generateContent?key=%s\" "
    "-H \"Content-Type: application/json\" "
    "-d @request.json > response.txt",
    API_KEY);

// Running the command via system call
int status = system(command);
```

```

if (status == 0) {
    // Open response.txt and parse the JSON result
    // ...
}

```

## Sample Output

```

=====
=====

FAST CALCULATOR PROJECT - GRAPHING MODE

=====
=====

```

Enter function of x: x\*x

Plotting:  $y = x \cdot x$

```

*      |      *
*      |      *
*      |      *
**     |     **
*****

```

## 5. Testing & Results

We tested everything against different scenarios to make sure it's stable, especially when it comes to the order of mathematical operations and connecting to the internet.

Everything worked the way it should. The math engine handled decimal precision correctly, and when the AI module had connection issues, it showed proper error messages instead of just crashing.

## 6. Conclusion, Limitations & References

### Conclusion

Our Smart Console Calculator really pushes what you'd normally expect from a first-semester C project. Instead of just hard-coding everything, we built dynamic expression parsing and connected to an external API, which shows we actually understand memory management, data structures, and how systems interact. It genuinely works as a "Super Calculator" that students can actually use.

### Limitations

- Needs Internet: The AI feature only works if you're online and have a valid API key.
- Low Graph Quality: Since we're drawing graphs with text characters (ASCII), the resolution is pretty low and wouldn't work for precise scientific measurements.
- Windows Only: We used windows.h for the UI, so the code won't work on Linux or MacOS without some changes.

### Future Enhancements

- Add a Real GUI: We could port this to C++ with Qt or use C# to build a proper graphical interface.

- Save History: Add file handling so past calculations and AI conversations get saved to a log file.
- Better Graphing: Include support for  $\sin()$ ,  $\cos()$ , and  $\tan()$  functions in graphing mode.

## References

- Documentation: Google Gemini API Docs (AI Studio).
- Algorithm: Edsger Dijkstra's Shunting-yard Algorithm (Wikipedia).
- Book: The C Programming Language by Brian Kernighan and Dennis Ritchie.
- Resources: StackOverflow community for help with curl command syntax.